ProvLoom: Explaining Agentic Skill Risk with Instruction and Runtime Provenance Chains

Anonymous Authors
Anonymous Institution

Abstract

Agentic skills package natural-language instructions, helper code, local resources, and tool-facing behavior into reusable units for LLM agents. Existing skill scanners are useful for surfacing suspicious indicators, but an analyst often needs a different answer: whether the available evidence supports a security-relevant path from a protected source, through concrete relays, to a sink, and where the path remains open. We present ProvLoom, a security triage and explanation framework for agentic skills. ProvLoom analyzes a skill through two bounded evidence views. Its static view loads local artifacts into span-grounded semantic units, extracts actions and entities, builds a typed instruction provenance graph, and validates instruction-level source-to-sink candidates. Its runtime view executes the skill in a Docker sandbox, normalizes file, process, network, tool, and LLM observations into RuntimeEvent records, propagates synthetic taint across supported carriers, builds a runtime provenance graph, and recovers evidence-graded chains. ProvLoom then reports policy violations, candidate chains, coverage gaps, and static-runtime alignment rather than presenting a single opaque maliciousness label. On Benchmark v3, an 800-sample suite with frozen private ground truth, ProvLoom achieves 81.7% accuracy, 88.2% precision, 73.1% recall, and 0.799 F1 on completed samples. It recovers complete chains for 61.6% of expected complete-chain cases and keeps the benign-lookalike false-positive rate at 0.0%. These results show that provenance-backed explanation exposes both confirmed risky flows and the limits of bounded skill execution.

1 Introduction

LLM agents increasingly connect model reasoning to tools, local files, network endpoints, and system commands [1, 21, 22, 30–32]. Reusable skills make this execution surface portable. A skill package can contain a `SKILL.md` file, setup notes, helper scripts, configuration files, local resources, and examples that guide how an agent or a human operator should invoke external capabilities. This packaging model is valuable because it turns repeated tasks into reusable procedures. It also creates a review problem: the security-relevant behavior of a skill may be split across natural-language instructions, scripts, generated files, tool calls, and model-selected actions.

Prior work on agent and skill security has shown that prompt injection, tool misuse, unsafe setup logic, overbroad capabilities, and supply-chain behavior can compromise agents that act through tools [2, 5, 7, 13, 14, 23, 26, 28]. Recent scanners help by labeling risky packages or emitting suspicious findings [2, 4, 14]. Such outputs are useful for candidate discovery, but they do not always answer the question an analyst faces after a finding appears. A sensitive file read and a network connection in the same trace do not by themselves establish exfiltration. A warning in documentation does not by itself show that the skill asks the agent to execute an unsafe path. A useful review artifact must explain whether the evidence actually closes a path.

This paper therefore separates two questions. The detection question asks whether a skill appears malicious or risky. The ProvLoom question asks whether available evidence supports a closed security-relevant path, and why. ProvLoom is not framed as a definitive malicious-skill classifier. It is a security triage and explanation framework that helps an analyst decide whether a candidate should be escalated, what source the risk begins from, which relays carry the effect, what sink or impact point is reached, and which parts of the explanation are supported by runtime evidence, instruction evidence, or only incomplete observations.

The key design challenge is that skill evidence has two different epistemic meanings. Static instruction evidence can show that a package asks an agent or operator to perform a risky sequence, even if bounded execution does not trigger it. Runtime evidence can show what happened in one execution, but bounded execution may miss latent setup behavior, nondeterministic model choices, unavailable external services, or encrypted payload contents. ProvLoom keeps these views separate until the assessment layer. It then reports confirmed runtime violations, static instruction paths, candidate depen-

dencies, alignment records, and coverage states rather than collapsing them into an unexplained risk score.

ProvLoom implements this design in a canonical pipeline. The static side loads local artifacts, parses them into semantic units with source locations, extracts normalized actions and entities, validates grounding, builds a typed instruction provenance graph, and checks bounded path templates for credential flows, download-execute behavior, persistence, permission expansion, destructive modification, reverse shells, resource abuse, and instruction-policy violations. The runtime side executes a skill in a Docker sandbox with a runtime wrapper and `strace`; it normalizes file, process, network, tool, and LLM observations; it seeds synthetic taint sources for protected resources; it propagates taint only across supported carriers; it builds a runtime provenance graph; and it recovers source-to-sink chains using evidence-aware path search.

We evaluate ProvLoom on Benchmark v3, a new 800-sample suite whose public skill bodies are separated from private ground truth, scenario cards, and fixtures. The benchmark is designed to test explanation behavior, including source recovery, relay recovery, sink recovery, complete-chain recovery, trusted-flow distinction, false-closure avoidance, coverage attribution, and static-runtime alignment. On 797 completed predictions, ProvLoom reaches 81.7% accuracy, 88.2% precision, 73.1% recall, and 0.799 F1. Its benign-lookalike false-positive rate is 0.0%, while trusted-allowed flows remain the main false-closure pressure with a 31.97% false-positive rate. The system recovers complete chains for 61.6% of samples that expect a complete chain, making the remaining gap visible rather than hidden.

This paper makes the following contributions.

- We recast skill analysis as provenance-backed security triage: the primary output is an evidence state over source, relay, and sink paths, not only a maliciousness label.
- We design and implement a hybrid analysis pipeline that combines span-grounded instruction provenance, carrier-aware runtime taint propagation, runtime provenance graphs, chain recovery, coverage certification, and static-runtime alignment.
- We construct Benchmark v3, an 800-sample benchmark with private ground truth, safe fixtures, counterfactual pairs, trusted allowed flows, benign lookalikes, and coverage-review cases.
- We evaluate ProvLoom and public baselines on Benchmark v3 under a same-corpus detection contract, while reporting chain-recovery metrics only for systems that emit chain evidence.

2 Background

2.1 Agentic Skills

An agentic skill is a local capability package for an LLM agent. The central artifact is usually a natural-language instruction file, but the effective package boundary can also include README files, scripts, configuration files, package metadata, examples, and local resources. These artifacts form two surfaces. The instruction surface tells an agent or operator what to do. The execution surface contains the code, commands, files, and external endpoints that may be touched when the skill runs.

This distinction matters because a risk may cross surfaces. A `SKILL.md` file may instruct an agent to read a token, write an intermediate report, and send that report to an endpoint. A helper script may perform the write or upload. A setup note may ask the operator to fetch and execute a remote installer. A runtime LLM may decide which tool to invoke and how to fill its arguments. Skill review therefore needs to preserve artifact boundaries and evidence locations while still reasoning over paths that cross documents, files, processes, tool calls, and network activity.

2.2 Static, Dynamic, and Taint Analysis

Static analysis inspects artifacts before execution. In the skill setting, static analysis must handle natural-language instructions as well as code and configuration. Dynamic analysis observes one or more executions. It can confirm behavior that actually occurred, but it is bounded by trigger coverage, sandbox configuration, model choices, service availability, and instrumentation visibility.

Taint analysis provides a useful vocabulary for the runtime part of the problem. A source introduces sensitive influence, propagation carries that influence through intermediate values or carriers, and a sink is an endpoint or operation where reaching it matters for a policy. The important point is negative as much as positive: a sensitive read and a network connection do not establish a data flow unless the analysis observes or reconstructs a supported propagation relation. For skills, propagation may pass through file contents, file paths, process arguments, environment variables, standard streams, pipes, tool arguments, tool returns, HTTP bodies, upload files, socket payloads, and LLM context. ProvLoom treats each carrier according to what the current instrumentation can actually observe.

2.3 Provenance Graphs and Source-to-Sink Paths

Provenance graphs represent how an outcome happened. Vertices model objects such as sources, files, processes, network endpoints, tool invocations, and instruction actions. Edges model relations such as reads, writes, executions, derived values, uploads, control triggers, or entity references. A source-to-sink path is closed when the graph contains a supported sequence from a protected source to a security-relevant sink under the analysis rules. A path remains open when evidence

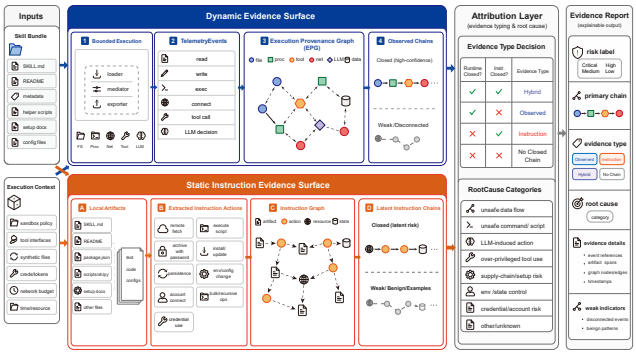

Figure 1: Placeholder overview of the ProvLoom pipeline. The USENIX version should redraw this figure to reflect Static v2, Dynamic v3, coverage certification, and static-runtime alignment.

is missing, weak, only temporally co-occurring, or outside the bounded execution.

Taint analysis asks whether sensitive influence reaches a sink. Provenance adds explanation: which carrier moved the value, which event supports an edge, which observation is weak, and where an analyst can trace the result back to original evidence. ProvLoom uses provenance-backed chains as its analyst-facing abstraction. Each chain records ordered nodes and edges, evidence references, missing observations, transformations, and a coverage state.

3 ProvLoom Detection Framework

3.1 Overview

Figure 1 gives the high-level flow. A skill enters ProvLoom as a local artifact bundle. The static analyzer produces instruction evidence: span-grounded actions, entities, an instruction provenance graph, validated static chains, and static coverage. The dynamic analyzer produces runtime evidence: normalized events, taint sources, a runtime provenance graph, recovered runtime chains, coverage, policy findings, and static-runtime alignment. The final assessment combines these outputs at the explanation layer.

The two evidence views are intentionally not fused into a single graph. Static instruction evidence means the package states or implies a path. Runtime evidence means an analyzed execution produced observations that may support a path. Alignment records relate the views by matching normalized files, endpoints, actions, and chains; they do not turn a static statement into runtime confirmation. This separation lets ProvLoom report runtime-confirmed, instruction-supported, runtime-only, instruction-only, candidate, and incomplete states without overstating any one evidence source.

3.2 Threat Model

ProvLoom targets third-party skills reviewed before use or during audit. The adversary can control skill artifacts, including instructions, helper code, package metadata, examples, and setup text. The adversary can try to hide behavior in natural language, generated files, tool arguments, process chains, or LLM-mediated steps. The adversary may depend on external services, credentials, delayed triggers, or nondeterministic model behavior.

The analysis environment provides a bounded sandbox, synthetic protected resources, local mock services for benchmark fixtures, and instrumentation over supported runtime surfaces. Real credentials are not provided. External platform state may be absent or mocked. Network access is configurable; the current benchmark full run uses controlled fixtures and recorded configuration rather than assuming the sandbox can safely interact with arbitrary services. ProvLoom does not claim to prevent malicious behavior. It reports what evidence was observed, what path was recovered, and where coverage prevents a stronger conclusion.

The audit goal is to decide whether available evidence supports a security-relevant chain and whether the case should be escalated. A confirmed runtime violation requires a concrete chain from a protected source to an unpermitted sink or an integrity-impact chain such as persistence. A candidate chain, timeout, environment failure, missing source, unavailable sink, or instrumentation gap yields review semantics rather than a benign proof.

4 The Design of ProvLoom

4.1 Local Artifact Loading

ProvLoom first determines the local artifact universe. The static loader starts from the skill root and skill file, prioritizes `SKILL.md`, README files, package and manifest files, and then recursively considers files within a bounded depth. The default configuration limits each file to 256KB, the total loaded content to 2MB, the depth to five, and the number of files to 160. It ignores version-control state, private ProvLoom adapter state, dependency directories, build outputs, vendor directories, and prior analysis artifacts. Supported artifacts include Markdown, text, JSON, YAML, TOML, INI-style configs, Python, shell, JavaScript, TypeScript, PowerShell, Dockerfiles, Makefiles, and common package files.

Each loaded file becomes a *static artifact* with a path, type, SHA-256 hash, size, encoding, and load status. Markdown is split into headings, paragraphs, list items, fenced code blocks, and command lines. Configuration files are split into field-level units, and code or text files are split into comments, assignments, function-call-like lines, command lines, or paragraphs. Every semantic unit stores artifact id, line range, byte offsets, parent section, language, and exact text. This repre-

sentation is deliberately simple, but it preserves traceability: later actions, entities, graph edges, and chains can point back to concrete source spans.

4.2 Static Instruction Chain Recovery

Natural-language instructions cannot be turned into chains by keyword co-occurrence alone. A secure warning, an example command, and a required action may all mention the same sensitive term. ProvLoom therefore extracts a typed representation before path validation.

The static pipeline uses deterministic extraction by default. The optional LLM extractor is disabled in the current default configuration; when enabled, it is constrained to span-grounded action extraction and semantic filtering. The normalized action schema contains an actor, action type, object mentions, source mentions, destination mentions, tool mentions, conditions, modality, evidence span, extractor provenance, grounding status, confidence, and validation notes. Action types include read, write, copy, move, delete, download, upload, send, execute, install, import, decode, extract, persist, access credential, invoke tool, invoke API, permission changes, collect, and transform. Modalities include required, recommended, optional, conditional, prohibited, example-only, descriptive, hypothetical, quoted-untrusted, and unknown.

The grounding validator checks that every retained action maps to a loaded artifact and semantic unit. The entity resolver groups mentions into entities and records whether a relation is confirmed, uncertain, inferred, or rejected. The instruction graph then creates document artifact nodes, evidence span nodes, entity nodes, and instruction action nodes. It adds SUPPORTEDBY edges from evidence spans to actions, action-specific edges such as READS, DOWNLOADS, EXECUTES, UPLOADS, SENDSTO, and PERSISTSAS, and entity-resolution edges such as SAMEENTITYAS, REFERSTO, and DERIVESFROM. This graph avoids false closure from mere shared words because path validation must pass through grounded actions and resolved entities rather than arbitrary text overlap.

The static path validator suppresses prohibited, example-only, hypothetical, quoted-untrusted, and descriptive modalities. It then checks bounded templates for credential paths, download-execute paths, droppers, persistence, destructive modification, permission expansion, reverse shells, ransomware-like behavior, resource abuse, and instruction-policy violations. A credential-exfiltration chain, for example, requires a sensitive read or collection, a send or upload action, an external sink, and a data-continuity relation tying the source to the payload or upload. If the sink is missing, the payload relation is absent, or the endpoint is only an authentication context, the chain is partial or capability-only rather than closed. Each static chain records status, source, sink, ordered nodes and edges, evidence units, unresolved links, conditions, policy status, alert status, data continuity, scope continuity, limitations, and review priority.

4.3 Sandboxed Dynamic Analysis

ProvLoom executes a skill in a Docker sandbox. The runner copies the skill to a temporary directory, mounts it inside the container, writes the input payload and LLM configuration, and starts the container with dropped Linux capabilities, `no-new-privileges`, PID, memory, and CPU limits. The container command runs `strace` over file, process, and network syscall classes while invoking the ProvLoom container runtime.

The runtime wrapper exposes virtual tools for reading files, writing files, running commands, and issuing HTTP requests. It also supports an LLM-mediated mode in which an OpenAI-compatible model selects tool calls over multiple rounds. Runtime logs record tool invocations and returns, LLM requests and responses, process outcomes, and selected metadata. For LLM requests, tainted context is represented by hash, byte count, and redacted preview rather than raw secret contents.

The trace parser extracts file events such as opens, writes, renames, and deletes; process events such as `execve`, fork, clone, and vfork; and network events such as connect and send-family syscalls. The normalizer merges these observations with runtime wrapper events, tool events, LLM events, data-flow hints, and taint events into timestamped RuntimeEvent records. A RuntimeEvent includes actor and object identity, operation, data hash or redacted preview, byte count, taint ids, evidence level, evidence strength, observation source, carrier type and location, raw reference, instrumentation visibility, and metadata.

4.4 Dynamic Chain Recovery

Runtime analysis begins by seeding protected sources. The default sensitive path policy includes files such as `/etc/passwd`, `/etc/shadow`, `/root/**`, private ProvLoom skill state, and adapter credential state. The taint registry creates synthetic markers of the form `PROVLOOM_SECRET_<id>_<entropy>` and registers raw, base64, hex, URL-encoded, JSON-escaped, and SHA-256 variants. Hash-derived matches are conservative evidence, not full payload observation.

The propagator processes events in timestamp order. A sensitive file read registers a source and taints the file and process input. Reading a tainted file propagates taint to the event and process input. Writing a file taints the target only when taint is explicit; prior sensitive process contact without output evidence becomes context, not confirmed content flow. File copy, move, rename, and extract operations inherit taint when the source file is known. Process execution detects markers in argv, environment, stdin, and command metadata; file taint becomes argument taint only when the event indicates file contents were passed. Pipes and standard streams propagate

taint when carrier data or marker evidence is observed; otherwise they record process-context candidates. Tool arguments and returns propagate structured taint references. Network sends and uploads propagate taint through explicit upload files or markers in headers, query strings, bodies, forms, multipart fields, socket payloads, or tool arguments. Opaque network payload after prior sensitive process contact remains a candidate context relation.

The runtime provenance graph contains SensitiveSource, Process, File, NetworkEndpoint, AgentSession, ToolInvocation, DataObject, RuntimeInstruction, and PersistenceTarget nodes. Edges include read, write, execute, fork, pipe, pass-as-argument, pass-as-env, return-to, control-trigger, send, connect, upload-file, persist, materialize-instruction, derive, and extract relations. For tainted carriers, ProvLoom creates DataObject nodes that represent the carrier and connects them through derive, propagate, send, or upload edges. Weak edges record process context and read-before-connect co-occurrence. Edges preserve event ids, taint ids, raw references, timestamps, transformations, evidence strengths, and instrumentation gaps.

Chain recovery performs bounded BFS from SensitiveSource nodes to terminal nodes. Confirmed confidentiality terminals are SENDS and UPLOADS edges to NetworkEndpoint nodes. Candidate confidentiality terminals include connect, co-occurrence, process-context, and legacy send or upload-file relations. Execution chains connect remote-artifact writes, derivations, or extracts to execution. Persistence chains end at persistence or materialized-instruction targets. The search uses a maximum path length of eight, enforces timestamp monotonicity, filters by terminal taint ids, and ranks paths by weak-edge count, evidence-strength penalty, whether data edges appear, and path length. A confirmed confidentiality chain cannot rely on hash-derived, process-context, temporal co-occurrence, candidate, or unknown evidence. Candidate chains require a potential carrier such as LLM context, HTTP fields, upload file, process argv, environment, stdin, socket payload, or tool argument.

4.5 Evidence Attribution

ProvLoom maps recovered evidence into a canonical assessment. A policy violation is reported when a confirmed confidentiality chain reaches a non-allowlisted sink, when persistence is confirmed, or when execution targets are outside the executable allowlist. Confirmed confidentiality chains are not treated as violations when the sink is trusted, the chain is only hash-derived, instrumentation gaps remain, the flow is trusted authentication, the LLM context is trusted, or an explicit permitted source-sink pair applies.

Coverage is a first-class output. Runtime-confirmed coverage means a confirmed or conservative chain was recovered without missing observations. Instrumentation-gap coverage means a chain exists but key payload or carrier visibility is incomplete. Insufficient coverage covers candidate dependencies or events without a closed canonical flow. Other states include timeout, max-steps-exhausted, execution-failed, environment-missing, sink-unavailable, path-not-triggered, target-reached-no-flow, and path-incomplete. The final decision contract uses `malicious` for confirmed policy violations, `needs_review` for incomplete or weak evidence states, and `benign` only when no violation or review condition is observed in the bounded analysis.

4.6 Security Considerations

The sandbox reduces host risk through container isolation, dropped capabilities, resource limits, and synthetic secrets, but it is not a full containment proof. The default network policy is configurable, and arbitrary external effects should be disabled or mocked for benchmark and predeployment audits. `strace` does not reliably expose encrypted payload contents; HTTP metadata is strongest when traffic goes through the runtime wrapper. LLM-mediated execution can be nondeterministic and may fail because of provider timeouts. ProvLoom therefore treats nonexecution, weak evidence, and missing carrier visibility as review states. The system is designed to make those limits visible rather than to guarantee complete detection.

5 The Design of Benchmark

Benchmark v3 evaluates whether an analysis system can recover security-relevant chains from realistic skill documentation and bounded executions. Detection metrics are included, but they are secondary to explanation tasks: source recovery, sink recovery, ordered edge recovery, complete-chain recovery, intermediate object and carrier recovery, minimal witness recovery, false-closure avoidance, coverage-state attribution, trusted-flow distinction, static-runtime alignment, and contradiction recovery.

The benchmark contains 800 natural-language Markdown skill samples. Tested skill bodies are public under `benchmark_v3/samples`; private scenario cards, fixtures, and ground-truth files are physically separated. The tested skills do not include structured `skill-actions` blocks that encode the answer. Ground truth uses an independent ontology rather than ProvLoom-internal schemas. Its fields include actor, protected asset, origin location, operation, transformation, intermediate object, destination, transport carrier, control condition, authorization context, ordered relation, expected observation, observability requirement, policy expectation, and minimal evidence set. Each complete chain is an ordered list of ontology object identifiers, and nonviolation samples record forbidden false chains or coverage conditions that explain why a chain must not close.

Table 1 summarizes the construction target and final distribution. The suite contains 400 confirmed violations, 198

Table 1: Benchmark v3 composition.

Category	Count
Total samples	800
Confirmed violation	400
Benign lookalike	198
Trusted allowed	122
Review coverage	80
Counterfactual pairs	160
LLM-mediated samples	327
Network or external-platform samples	600
Multi-file samples	200

benign lookalikes, 122 trusted-allowed flows, and 80 review-coverage cases. It spans 13 risk families, including credential exfiltration, private data upload, LLM-mediated disclosure, untrusted download and execute, persistence, privilege expansion, destructive modification, reverse shell, supply-chain abuse, unauthorized external actions, resource abuse, instruction override, and multi-stage behavior. It also contains 160 counterfactual pairs, 327 LLM-mediated samples, 600 network or external-platform samples, and 200 multi-file samples.

Benchmark v3 is designed to reduce benchmark bias in three ways. First, ground truth is written and stored separately from tested skill text, and it is frozen before ProvLoom or baselines are evaluated. Second, benign lookalikes and trusted-allowed cases deliberately resemble risky workflows while changing authorization, source identity, sink trust, or policy conditions. These cases test whether a system closes a path only when the protected source and unpermitted sink relation is present. Third, fixtures use synthetic assets, sandbox paths, and local or mock-only services, so dynamic replay does not require real credentials or external side effects.

The construction report records 800 complete skill samples, 800 scenario cards, 800 private ground-truth files, and 800 safe fixture files. Validation passed for all 800 samples, and representative deterministic replays across early, middle, and late samples returned exit code 0 without timeouts. The split is development=120, blind-heldout=480, and challenge-heldout=200; the split hash begins with `ce6a2473`. All skill bodies are 701–922 words.

Evaluation boundary. The private ground truth is used only by evaluation scripts, not by the analyzer. The metrics artifact explicitly records `ground_truth_loaded_by_analyzer=false`. This boundary matters because ProvLoom has path templates and policy rules; using ground truth to tune predictions or repair baseline mappings would invalidate the comparison.

Table 2: ProvLoom results on Benchmark v3 completed samples.

Metric	Value
Completed predictions	797
Failed samples	3
Accuracy	0.8168
Precision	0.8818
Recall	0.7312
F1	0.7995
False-positive rate	0.0977
Benign-lookalike FPR	0.0000
Trusted-allowed FPR	0.3197
Review rate	0.1368
Confirmed violation chain recall	0.6181
Complete-chain recall	0.6156

6 The Evaluation of ProvLoom

We evaluate three questions. First, can ProvLoom triage risky and benign skills on the same Benchmark v3 corpus used for baselines? Second, can it recover complete source-to-sink chains rather than only labels? Third, where do failures come from when evidence does not close?

6.1 Setup

The main ProvLoom run is stored in the artifact directory `benchmark_v3_full_glm52_steps16_0001_0800`. It used model `glm-5.2` with Docker image `skill-runtime-sandbox:dynamic-v3`. The evaluation metrics were generated from `metrics.json`; the prediction hash begins with `6bb33ae4`. Three samples failed at the run level, so detection metrics are reported on 797 completed predictions while still recording 800 evaluated samples.

6.2 End-to-End Results

Table 2 reports the main result. ProvLoom reaches 81.7% accuracy, 88.2% precision, 73.1% recall, and 0.799 F1. It reports 330 malicious and 467 benign binary predictions over completed samples. The false-positive rate is 9.77%. The benign-lookalike false-positive rate is 0.0%, which indicates that ProvLoom usually avoids closing paths in documentation that resembles a risky flow but lacks the protected-source to unpermitted-sink relation. The trusted-allowed false-positive rate is 31.97%, showing that trust and authorization distinctions are the current largest precision weakness.

Outcome-level results explain the aggregate numbers. On confirmed violations, ProvLoom finds 291 true positives and misses 107, giving 73.1% recall. On benign lookalikes, it produces 197 true negatives and no false positives. On review-coverage cases, it produces 80 true negatives with a 20.0%

Table 3: Runtime coverage states in the full Benchmark v3 run.

Coverage state	Count
complete	290
path_incomplete	325
execution_failed	87
timeout	41
environment_missing	25
target_reached_no_flow	21
path_not_triggered	5
max_steps_exhausted	3
run-level failed	3

review rate. On trusted-allowed cases, it produces 83 true negatives and 39 false positives. The last category is not a generic benign failure: the workflows can contain real data movement to trusted or authorized destinations, and the system currently over-escalates a portion of those flows.

6.3 Chain Recovery and Coverage

ProvLoom recovered complete chains for 61.6% of expected complete-chain cases and confirmed violation chains for 61.8%. These metrics are lower than detection precision because the chain contract is stricter. A label can be correct when a risk is visible but the minimal source, relay, and sink path is incomplete; the chain metric only credits cases whose expected complete-chain evidence is recovered.

Coverage states make the failure boundary visible. Among the 797 completed predictions, 290 ended in complete coverage, 325 in path-incomplete coverage, 87 in execution-failed coverage, 41 in timeout, 25 in environment-missing, 21 in target-reached-no-flow, five in path-not-triggered, and three in max-steps-exhausted. Three additional samples failed before producing normal summary fields. This distribution shows why ProvLoom should not be described as a definitive detector: many cases are bounded-execution limitations or unresolved path states, not confirmed absence of risk.

6.4 Split Results

The development split scores higher than the held-out splits, as expected. On development samples, ProvLoom reaches 89.9% accuracy and 0.897 F1. On blind-heldout samples, it reaches 80.3% accuracy and 0.780 F1. On challenge-heldout samples, it reaches 80.0% accuracy and 0.783 F1. Complete-chain recall is 85.0% on development, 58.0% on blind-heldout, and 56.0% on challenge-heldout. The gap suggests that chain recovery is sensitive to more complex held-out phrasing, trusted-flow distinctions, and execution coverage.

6.5 Baseline Comparison

Table 4 compares systems on the same Benchmark v3 corpus under a binary detection contract. Chain metrics are only reported for ProvLoom because the listed baselines emit labels, findings, or severity-like outputs rather than source-relay-sink chain objects. We therefore do not assign them zero chain score.

The baseline configuration audit prevents two misleading comparisons. The Sentry row is not a full Sentry Skill Scanner workflow: the artifact invokes the bundled deterministic static script and no LLM-backed full review. The SkillScan row is partial or audit-dominant: provider detection did not yield usable native LLM execution in the child environment, and Docker sandbox evidence did not contribute. We therefore label both rows by their actual artifact mode. Cisco's LLM-enabled scanner has the strongest binary F1 in the current artifacts; ProvLoom's distinct contribution is that it exports explanation state and chain evidence in addition to a label.

6.6 Ablation Status

The repository contains ablation artifacts under `artifacts/benchmark_v3_ablation` for event-only, static-only, no-alignment, and no-policy variants. These artifacts are useful for the next revision, but this draft does not yet include a fully audited ablation table.

7 Real-world Audit of Skill Ecosystem

The current repository contains legacy real-world audit materials and manually produced review artifacts, but this USENIX draft does not yet have a newly audited real-world result that is aligned with Benchmark v3 and Dynamic v3. We therefore do not reuse the ACSAC numbers as if they were current results.

The intended role of the real-world section is to evaluate triage rather than to claim a definitive ecosystem prevalence estimate. A sound protocol should state the collection source, deduplication and sampling process, prescreening method, manual-review rubric, reviewer agreement process, responsible-disclosure status, and how ProvLoom evidence was used. It should also distinguish system-generated queues from manual ground truth: ProvLoom can prioritize and explain candidate paths, while human reviewers decide whether the evidence and context justify a confirmed malicious label.

For the next data refresh, the analysis should report the composition of the need-review and high-risk queues, the precision of confirmed queues, the number of instruction-only, runtime-confirmed, hybrid, and open-chain cases, and failure causes for confirmed malicious samples without closed runtime chains. Until that audit is complete, this section is intentionally conservative.

Table 4: Detection comparison on Benchmark v3. Chain metrics are reported only when the system emits chain evidence.

System	Acc.	Prec.	Rec.	F1	FPR	BL-FPR	TA-FPR	Failed
ProvLoom	0.8168	0.8818	0.7312	0.7995	0.0977	0.0000	0.3197	3
AI-Infra-Guard	0.6208	0.9533	0.2550	0.4024	0.0125	0.0102	0.0246	1
Cisco LLM scanner	0.8440	0.8920	0.7850	0.8351	0.0962	0.0816	0.1833	5
Sentry bundled static script	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0
SkillScan partial scan	0.6510	0.8889	0.3600	0.5125	0.0468	0.0219	0.0738	15

8 Related Work

8.1 Agent and Skill Security

Agent security work studies how untrusted instructions and model decisions affect tools, files, APIs, and external actions [1, 5, 7, 13, 23, 26, 27, 31]. Skill security work focuses on reusable packages that combine instructions, code, setup behavior, and capability descriptions [2, 11, 14, 20, 28]. These studies motivate ProvLoom's artifact boundary: risk can appear in prose, helper code, package metadata, or runtime tool use.

SkillScan, Cisco AI Defense Skill Scanner, SkillFortify, and related tools are useful for scanning skill packages and surfacing risky indicators [2, 4, 14]. Semia is especially close in spirit because it lifts natural-language and interface artifacts into a structured representation before deterministic reachability checks [28]. ProvLoom differs in analysis objective. It focuses on analyst-reviewable provenance chains and explicitly separates static instruction evidence from runtime evidence and coverage states.

8.2 Runtime Security, Detonation, and Benchmarks

Dynamic agent and skill analysis systems execute or simulate artifacts to expose behavior that static scans may miss. This direction is complementary to static instruction analysis because runtime evidence can show actual tool calls, file writes, process execution, and network effects. ProvLoom follows this line but avoids treating execution co-occurrence as flow evidence. It requires a supported carrier or an explicit weak-state report before closing a chain.

Benchmarks for agent and skill security define the workload and ground-truth contract that make evaluations comparable [5, 32]. Benchmark v3 follows the same high-level motivation but targets explanation-specific tasks. Its ground truth records ordered relations, minimal evidence sets, forbidden false chains, trusted-flow distinctions, and coverage expectations.

8.3 Taint Analysis and Provenance

Static taint, value-flow, and code property graph systems reason about source-to-sink paths in code [15, 24, 25, 29]. System provenance research reconstructs causal histories from process, file, socket, and audit events for intrusion investigation and threat hunting [3, 6, 8–10, 12, 16–19]. ProvLoom borrows the source-to-sink and provenance vocabulary, but the object of analysis differs. Skill evidence spans prose, code, tool wrappers, LLM context, generated files, and sandbox events. The contribution is not a new whole-system provenance collector; it is a skill-layer explanation system that reports which evidence view supports each chain.

9 Discussion

ProvLoom's current strengths and weaknesses are visible in the evaluation. The system is good at avoiding false closure on benign lookalikes, which supports the decision to require grounded source-to-sink relations rather than keyword co-occurrence. It is weaker on trusted-allowed flows, where the same data movement can be benign or risky depending on authorization and sink trust. This weakness is partly a policy-model problem and partly an evidence problem: some flows expose a carrier but not enough context to decide whether the destination is authorized.

The main technical boundary is coverage. Dynamic analysis observes one bounded execution. If a skill requires real credentials, an unavailable command, a delayed maintenance trigger, a particular LLM decision, or an external service that is not mocked, ProvLoom cannot honestly report a runtime-confirmed chain. The current design treats these cases as review states, which may lower binary recall but gives the analyst a more faithful explanation.

Static analysis also has limits. The default extractor is deterministic and span-grounded, which improves reproducibility, but it cannot understand every paraphrase, implication, or adversarially written instruction. Optional LLM extraction exists, but it is disabled by default in the current configuration and must remain span-grounded and auditable before it can be used for claims in this paper.

Finally, ProvLoom still reports a binary prediction for benchmark compatibility. The paper's central claim should not be read from that field alone. The more important artifact is the explanation bundle: static chains, runtime chains, alignment records, policy findings, coverage certificate, minimal witnesses, and limitations.

10 Conclusion

ProvLoom reframes skill security analysis as evidence-backed triage and explanation. It combines span-grounded static instruction provenance with carrier-aware runtime provenance to decide whether a protected source reaches a security-relevant sink under bounded analysis. On Benchmark v3, ProvLoom provides competitive detection metrics while exposing complete chains, false closures, and coverage gaps that a label-only scanner cannot express.

References

[1] AN, H., ZHANG, J., DU, T., ZHOU, C., LI, Q., LIN, T., AND JI, S. IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Suzhou, China, Nov. 2025), Association for Computational Linguistics, pp. 1023–1039.

[2] BHARDWAJ, V. P. Formal analysis and supply chain security for agentic AI skills. *arXiv preprint arXiv:2603.00195* (2026).

[3] CHENG, Z., LV, Q., LIANG, J., WANG, Y., SUN, D., PASQUIER, T., AND HAN, X. KAIROS: Practical intrusion detection and investigation using whole-system provenance. In *Proceedings of the IEEE Symposium on Security and Privacy* (2024).

[4] CISCO AI DEFENSE. Cisco AI Defense Skill Scanner. https://github.com/cisco-ai-defense/skill-scanner, 2026. Accessed: 2026-05-13.

[5] DEBENEDETTI, E., ZHANG, J., BALUNOVIĆ, M., BEURER-KELLNER, L., FISCHER, M., AND TRAMÈR, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37* (2024).

[6] GAO, P., XIAO, X., LI, Z., JEE, K., XU, F., KULKARNI, S. R., AND MITTAL, P. AIQL: Enabling efficient attack investigation from system monitoring data. In *Proceedings of the 2018 USENIX Annual Technical Conference* (2018).

[7] GRESHAKE, K., ABDELNABI, S., MISHRA, S., ENDRES, C., HOLZ, T., AND FRITZ, M. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173* (2023).

[8] HAN, X., PASQUIER, T., BATES, A., MICKENS, J., AND SELTZER, M. UNICORN: Runtime provenance-based detector for advanced persistent threats. In *Proceedings of the Network and Distributed System Security Symposium* (2020).

[9] HASSAN, W. U., GUO, S., LI, D., CHEN, Z., JEE, K., LI, Z., AND BATES, A. NoDoze: Combatting threat alert fatigue with automated provenance triage. In *Proceedings of the Network and Distributed System Security Symposium* (2019).

[10] HOSSAIN, M. N., MILAJERDI, S. M., WANG, J., ESHETE, B., GJOMEMO, R., SEKAR, R., STOLLER, S., AND VENKATAKRISHNAN, V. N. SLEUTH: Real-time attack scenario reconstruction from COTS audit data. In *Proceedings of the 26th USENIX Security Symposium* (2017).

[11] JIA, X., LIAO, J., QIN, S., GU, J., REN, W., CAO, X., LIU, Y., AND TORR, P. SkillJect: Effectively automating skill-based prompt injection for skill-enabled agents. *arXiv preprint arXiv:2602.14211* (2026).

[12] KING, S. T., AND CHEN, P. M. Backtracking intrusions. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles* (2003).

[13] LIU, Y., JIA, Y., GENG, R., JIA, J., AND GONG, N. Z. Formalizing and benchmarking prompt injection attacks and defenses. In *Proceedings of the 33rd USENIX Security Symposium* (2024).

[14] LIU, Y., WANG, W., FENG, R., ZHANG, Y., XU, G., DENG, G., LI, Y., AND ZHANG, L. Agent skills in the wild: An empirical study of security vulnerabilities at scale. *arXiv preprint arXiv:2601.10338* (2026).

[15] LIVSHITS, V. B., AND LAM, M. S. Finding security vulnerabilities in java applications with static analysis. In *Proceedings of the 14th USENIX Security Symposium* (2005).

[16] MA, S., ZHANG, X., AND XU, D. ProTracer: Towards practical provenance tracing by alternating between logging and tainting. In *Proceedings of the Network and Distributed System Security Symposium* (2016).

[17] MILAJERDI, S. M., ESHETE, B., GJOMEMO, R., AND VENKATAKRISHNAN, V. N. POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security* (2019).

[18] MILAJERDI, S. M., GJOMEMO, R., ESHETE, B., SEKAR, R., AND VENKATAKRISHNAN, V. N. HOLMES: Real-time APT detection through correlation of suspicious information flows. In *Proceedings of the IEEE Symposium on Security and Privacy* (2019).

[19] PASQUIER, T., HAN, X., MOYER, T., BATES, A., HERMANT, O., EYERS, D., BACON, J., AND SELTZER, M. Runtime analysis of whole-system provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (2018).

[20] SAHA, S., FAGHIH, K., AND FEIZI, S. Under the hood of `SKILL.md`: Semantic supply-chain attacks on AI agent skill registry. *arXiv preprint arXiv:2605.11418* (2026).

[21] SCHICK, T., DWIVEDI-YU, J., DESSÌ, R., RAILEANU, R., LOMELI, M., HAMBRO, E., ZETTLEMOYER, L., CANCEDDA, N., AND SCIALOM, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36* (2023).

[22] SHEN, Y., SONG, K., TAN, X., LI, D., LU, W., AND ZHUANG, Y. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems 36* (2023).

[23] SHI, J., YUAN, Z., TIE, G., ZHOU, P., GONG, N. Z., AND SUN, L. Prompt injection attack to tool selection in LLM agents. In *Proceedings of the Network and Distributed System Security Symposium* (2026).

[24] SUI, Y., AND XUE, J. SVF: Interprocedural static value-flow analysis in LLVM. In *Proceedings of Compiler Construction* (2016).

[25] TRIPP, O., PISTOIA, M., FINK, S. J., SRIDHARAN, M., AND WEISMAN, O. TAJ: Effective taint analysis of web applications. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation* (2009), PLDI '09, ACM, pp. 87–97.

[26] WALLACE, E., XIAO, K., LEIKE, R., WENG, L., HEIDECKE, J., AND BEUTEL, A. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208* (2024).

[27] WANG, P., LIU, Y., LU, Y., CAI, Y., CHEN, H., YANG, Q., ZHANG, J., HONG, J., AND WU, Y. AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection. *arXiv preprint arXiv:2508.01249* (2025).

[28] WEN, H., LI, Y., LIU, H., SHOU, C., CHEN, Y., TIAN, Y., AND FENG, Y. Semia: Auditing agent skills via constraint-guided representation synthesis. *arXiv preprint arXiv:2605.00314* (2026).

[29] YAMAGUCHI, F., GOLDE, N., ARP, D., AND RIECK, K. Modeling and discovering vulnerabilities with code property graphs. In *Proceedings of the IEEE Symposium on Security and Privacy* (2014).

[30] YAO, S., ZHAO, J., YU, D., DU, N., SHAFRAN, I., NARASIMHAN, K., AND CAO, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations* (2023).

[31] ZHAN, Q., LIANG, Z., YING, Z., AND KANG, D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024* (2024).

[32] ZHANG, H., HUANG, J., MEI, K., YAO, Y., WANG, Z., ZHAN, C., WANG, H., AND ZHANG, Y. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. *arXiv preprint arXiv:2410.02644* (2024).